Lecture 1:
ML Systems

***Q:** What is a Machine Learning (ML) System?*

A data processing system (aka data system) for mathematically advanced data analysis operations (inferential or predictive):

❖ Statistical analysis; ML, deep learning (DL); data mining (domain-specific applied ML + feature eng.); High-level APIs to express ML computations over (large) datasets; Execution engine to run ML computations efficiently

Categorizing ML Systems

Orthogonal Dimensions of Categorization:

1. Scalability: In-memory libraries vs Scalable ML system (works on larger-than-memory datasets)

2. Target Workloads: General ML library vs Decision tree-oriented vs Deep learning, etc.

3. Implementation Reuse: Layered on top of scalable data system vs Custom from-scratch framework

Key concerns in ML: Accuracy, Runtime efficiency (sometimes), Additional key practical concerns in ML Systems: Scalability (and efficiency at scale), Usability, Manageability, Developability, Explainability

---

Lecture 2: Computer Organization

***Q:** What is a computer?*

A programmable electronic device that can store, retrieve, and process digital data.

Parts of a Computer

Hardware: The electronic machinery (wires, circuits, transistors, capacitors, devices, etc.)

Software: Programs (instructions) and data

Key Parts of Computer Hardware

Processor (CPU, GPU, etc.)

❖ Hardware to orchestrate and execute instructions to manipulate data as specified by a program

Main Memory (aka Dynamic Random Access Memory)

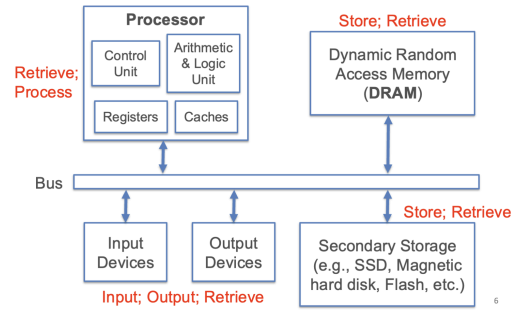
❖ Hardware to store data and programs that allows very fast location/retrieval; byte-level addressing scheme

Disk (aka secondary/persistent storage)

❖ Similar to memory but persistent, slower, and higher capacity / cost ratio; various addressing schemes

Network interface controller (NIC)

❖ Hardware to send data to / retrieve data over network of interconnected computers/devices



Key Aspects of Software

❖ **Instruction:** A command understood by hardware; finite vocabulary for a processor: Instruction Set Architecture (ISA); bridge between hardware and software

❖ **Program** (aka **code**): A collection of instructions for hardware to execute

❖ **Programming Language (PL):** A human-readable formal language to write programs; at a much higher level of abstraction than ISA

❖ **Application Programming Interface (API):** A set of functions ("interface") exposed by a program/set of programs for use by humans/other programs

❖ **Data:** Digital representation of information that is stored, processed, displayed, retrieved, or sent by a program

Main Kinds of Software

❖ **Firmware:** Read-only programs "baked into" a device to offer basic hardware control functionalities

❖ **Operating System (OS):** Collection of interrelated programs that work as anintermediary platform/service to enable application software to use hardware more effectively/easily. *Examples: Linux, Windows, MacOS, etc.*

❖ **Application Software:** A program or a collection of interrelated programs to manipulate data, typically designed for human use. *Examples: Excel, Chrome, PostgreSQL, etc.*

Bits: All digital data: Sequences of 0 & 1 (binary digits).

❖ Physics! Electronic Implementation

❖ Easy to store with bistable elements. e.g., high-low/off-on electromagnetism on disk.

❖ Reliably transmitted on noisy and inaccurate wires

❖ Very expressive and efficient.

❖ **Qbits**: More (and different!) physics behind quantum computing: electron spin up (1), spin down (0) ...and every state in between

Bits: All digital data are sequences of 0 & 1 (binary digits)

Layers of abstraction to interpret bit sequences

Data type: First layer of abstraction to interpret a bit sequence with a human-understandable category of information; interpretation fixed by the programming language (PL).

❖ Example common datatypes: Boolean, Byte, Integer, "floating point" number (Float), Character, and String

Data structure: A second layer of abstraction to organize multiple instances of same or varied data types as a more complex object with specified properties

❖ Examples: Array, Linked list, Tuple, Graph, etc.

The size and interpretation of a data type depends on PL

A **Byte** (B; 8 bits) is typically the basic unit of data types

Boolean: Examples in data science: Y/N or T/F responses

❖ Just 1 bit needed but actual size is almost always 1B, i.e., 7 bits are wasted! (Q: Why?)

Integer: Examples in data science: #friends, age, #likes

❖ Typically 4 bytes; many variants (short, unsigned, etc.)

❖ Java int can represent -231 to (231 - 1);

❖ C unsigned int can represent 0 to (232 - 1);

❖ Python int is effectively unlimited length (PL magic!)

Hexadecimal representation is a common stand-in for binary representation; more succinct and readable

❖ Base 16 instead of base 2 cuts display length by ~4x

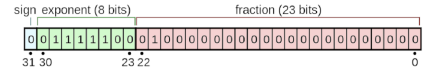
❖ Digits are 0, 1, ... 9, A (10_{10}), B, ... F (15_{10})

❖ To convert a decimal to hexadecimal, it might be convenient to make a diversion by converting to binary first:

❖ From binary, combine 4 bits at a time starting from lowest-value digits

i.e: 163_{10} = 1010 0011$_2$ = $A3_{16}$ or 0xA3 or $A3_H$

Float: Examples in data science: salary, scores, model weights; IEEE-754 single-precision format is 4B long; double-precision format is 8B long; Java and C float is single; Python float is double!



Decimal representation: $(-1)^{sign} \times 2^{exponent-127} \times (1 + \sum_{i=1}^{23} b_{23-i} 2^{-i})$

In above, exponent = $(1111100)_2 = (124)_{10}$, so:

$$(-1)^0 \times 2^{124-127} \times (1 + 1 \cdot 2^{-2})$$

$$= (1/8) \times (1 + (1/4)) = 0.15625$$

Due to representation imprecision issues, floating point arithmetic (addition and multiplication) is not associative!

❖ In binary32, special encodings recognized:

❖ Exponent 0xFF and fraction 0 is +/- "Infinity"

❖ Exponent 0xFF and fraction <> 0 is "NaN"

❖ Max is ~ 3.4 x 10^{38}; min +ve is ~ 1.4 x 10^{-45}

❖ More float standards: double-precision (float64; 8B) and half-precision (float16; 2B); different #bits for exponent, fraction

❖ Float16 is now common for deep learning parameters:

❖ Native support in PyTorch, TensorFlow, etc.; APIs also exist for weight quantization/rounding post training

❖ NVIDIA Deep Learning SDK support mixed-precision training; 2-3x speedup with similar accuracy!

❖ New processor hardware (FPGAs, ASICs, etc.) enable arbitrary precision, even 1-bit (!), but accuracy is lower

Representing **Character (char)** and **String**:

❖ Represents letters, numerals, punctuations, etc.

❖ A string is typically just a variable-sized array of char

❖ C char is 1 byte; Java char is 2 bytes; Python does not have a char type (use str or bytes)

❖ American Standard Code for Information Interchange (ASCII) for encoding characters; initially 7-bit; later extended to 8-bit; Examples: 'A' is 61, 'a' is 97, '@' is 64, '!' is 33, etc.

❖ Unicode UTF-8 is now most common; subsumes ASCII; 4 bytes for ~1.1 million "code points" incl. many other language scripts, math symbols, emojis, etc.

Serialization and Deserialization:

❖ A data structure often needs to be persisted (stored in a file) or transmitted over a network

❖ **Serialization** is the process of converting a data structure (or program objects in general) into a neat sequence of bytes that can be exactly recovered; **deserialization** is the reverse, i.e., bytes to data structure

❖ Serializing bytes and characters/strings is trivial

2 alternatives for serializing integers/floats:

❖ As *byte stream* (aka "binary type" in SQL)

❖ As *string*, e.g., 4B integer 5 -> 2B string as "5"

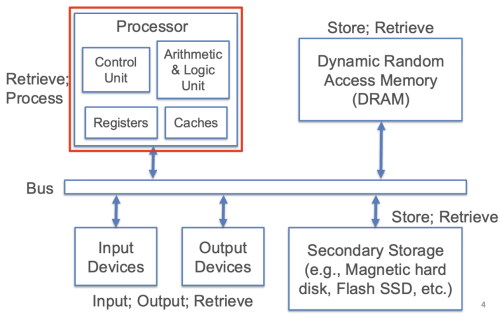
❖ String serialization common in data science (CSV, etc.)

Lecture 4: Processors and Memory Hierarchy

Processor: Hardware to orchestrate and execute instructions to manipulate data as specified by a program

❖ Examples: CPU, GPU, FPGA, TPU, embedded, etc.

Instruction Set Architecture (ISA): The vocabulary of commands of a processor; Program in PL -> Compile/Interpret -> Program in Assembly Language -> Assemble -> Machine code tied to ISA -> Run on processor



***Q:** How does a processor execute machine code?*

❖ Most common approach: **load-store architecture**

❖ **Registers**: Tiny local memory ("scratch space") on processor into which instructions and data are copied

❖ Instruction Set Architecture specifies bit length/format of machine code commands

❖ Instruction Set Architecture has several commands to manipulate register contents

Types of ISA commands to manipulate register contents:

❖ **Memory access**: **load** (copy bytes from DRAM address to register); **store** (reverse); put constant

❖ **Arithmetic & logic** on data items in registers: add/multiply/etc.; bitwise ops; compare, etc.

❖ **Control flow** (branch, call, etc.)

❖ **Caches**: Small local memory to buffer instructions/data

Processor Performance

***Q:** How fast can a processor process a program?*

Modern CPUs can run millions of instructions per second!

❖ ISA tells us **#clock cycles** each instruction needs

❖ CPU's **clock rate** lets us convert that to runtime (ns)

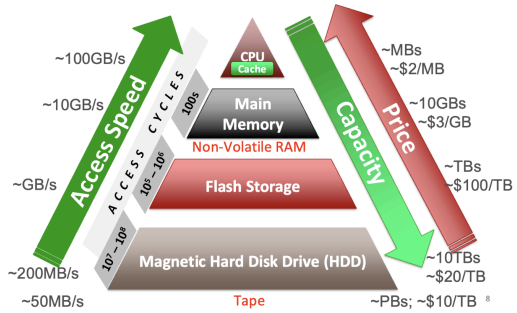
Alas, most programs do not keep CPU always busy

❖ Memory access commands **stall** the processor; Arithmetic & Logic Unit and Control Unit are idle during memory-register transfer

❖ Worse, data may not be in DRAM—wait for disk I/O!

❖ So, actual *execution runtime* of program may be orders of magnitude higher than what clock rate calculation suggests

Key Principle: Optimizing access to main memory and use of processor cache is critical for processor performance!



Key Principle: Locality of Reference

Carefully handling/optimizing access to main memory and use of processor cache is critical for processor performance!

Due to out-of-memory (OOM) access latency differences across memory hierarchy, optimizing access to lower levels and careful use of higher levels is critical for overall system performance!

Locality of Reference: Many programs tend to access memory locations in a somewhat predictable manner

❖ **Spatial**: Nearby locations will be accessed soon

❖ **Temporal**: Same locations accessed again soon

Locality can be exploited to reduce runtimes using **caching** and/or **prefetching** across all levels in the hierarchy

Data Layout: The order in which data items of a complex data structure or an abstract data type (ADT) are laid out in memory/disk

Data Access Pattern (of a program on a data object): The order in which a program has to access items of a complex data structure/ADT in memory

Hardware Efficiency (of a program):

❖ How close *actual execution runtime* is to best possible runtime given the processor clock rate and ISA

❖ Improved with careful data layout of all data objects used by a program based on its data access patterns

❖ Key Principle: Raise cache hits; reduce memory stalls!

Memory Management

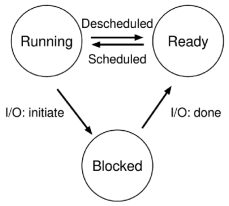
Caching: Buffering a copy of bytes (instructions and/or data) from a lower level at a higher level to exploit locality

Prefetching: Preemptively retrieving bytes (typically data) from addresses not explicitly asked yet by program

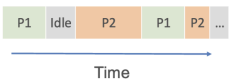
Spill/Miss/Fault: Data needed for program is not yet available at a higher level; need to get it from lower level
❖ **Register Spill** (register to cache);
❖ **Cache Miss** (cache to main memory);
❖ **Page Fault** (main memory to disk)
Hit: Data needed is already available at higher level
Cache Replacement Policy: When new data needs to be loaded to higher level, which old data to evict to make room? Many policies exist with different properties.

Lecture 5: Operating Systems

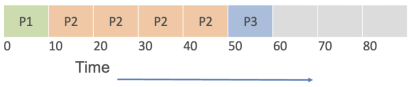
An OS is a large set of interrelated programs that make it easier for applications and user-written programs to use computer hardware *effectively, efficiently, and securely*
❖ Analogue ex: Consider the government's role in a country. Without OS, computer users must speak machine code!
2 key principles in OS (any system) design & implementation:
❖ **Modularity**: Divide system into functionally cohesive components that each do their jobs well
❖ Gov't ex: Consider executive-legislature-judiciary split
❖ **Abstraction**: *Layers of functionalities* from low-level (close to hardware) to high level (close to user)
❖ Gov't ex: Consider local-city-county-state-federal levels.
❖ **Kernel**: The core of an OS with modules to abstract the hardware and APIs for programs to use
❖ Auxiliary parts of OS include shell/terminal, file browser for usability, extra programs installed by I/O devices, etc.
Process: A running program, the central abstraction in OS
❖ Started by OS when a program is executed by user
❖ OS keeps inventory of "alive" processes (**Process List**) and handles apportioning of hardware among processes
***Q:** Why bother knowing process management?*
❖ A query is a program that becomes a process
❖ A data system typically abstracts away process management because user specifies the queries / processes in system's API
High-level steps OS takes to get a process going:
1. Create a process (get Process ID; add to Process List)
2. Assign part of DRAM to process, aka its Address Space
3. Load code and static data (if applicable) to that space
4. Set up the inputs needed to run program's main()
5. Update process' State to Ready
6. When process is scheduled (Running), OS temporarily hands off control to process to run the show!
7. Eventually, process finishes or run Destroy
***Q:** But is it not risky/foolish for OS to hand off control of hardware to a process (random user-written program)?!*
❖ OS has *mechanisms* and *policies* to regain control
Virtualization: Each hardware resource is treated as a virtual entity that OS can divide up and share among processes in a controlled way
Limited Direct Execution:
❖ OS mechanism to time-share CPU and preempt a process to run a different one, aka "context switch"
❖ A Scheduling policy tells OS what time-sharing to use
❖ Processes also must transfer control to OS for "privileged" operations (e.g., I/O); System Calls API
Virtualization of Processors
❖ Virtualization of processor enables process **isolation**, i.e., each process given an "illusion" that it alone runs
❖ Inter-process communication possible in System Calls API
❖ Later: Generalize to **Thread** abstraction for **concurrency**
Process Management by OS
OS keeps moving processes between 3 states:



❖ Gantt Chart: A vizualization: Shows what process runs when (on processor)

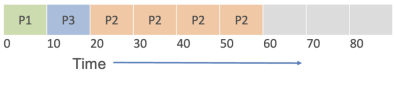


❖ Sometimes, if a process gets "stuck" and OS did not schedule something else, system **hangs**; need to reboot!
Scheduling Policies/Algorithms
Schedule: Record of what process runs on each CPU&when
Policy controls how OS time-shares CPUs among processes
Key terms for a process (aka **job**):
❖ **Arrival Time**: Time when process gets created
❖ **Job Length**: Duration of time needed for process
❖ **Start Time**: Times when process first starts on processor
❖ **Completion Time**: Time when process finishes/killed
❖ **Response Time** = [Start Time] – [Arrival Time]
❖ **Turnaround Time** = [Completion Time] – [Arrival Time]
Workload: Set of processes, arrival times, and job lengths that OS Scheduler has to handle
❖ In general, OS may not know all Arrival Times and Job Lengths beforehand!
❖ **Preemption** is possible: It is the act of the scheduler temporarily interrupting a running task so that another runs
❖ Key Principle: Inherent tension in scheduling between overall workload performance and allocation fairness
❖ Performance metric is usually *Average Turnaround Time*
❖ Fairness: Many metrics exist (e.g., Jain's fairness index)
Scheduling Policy: FIFO
❖ First-In-First-Out aka First-Come-First-Served (FCFS)
❖ Ranking criterion: Arrival Time; no preemption
Example: P1, P2, P3 of lengths 10,40,10 units arrive closely in that order



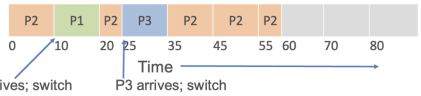
Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	10	0	10
P2	0	10	50	10	50
P3	0	50	60	50	60
				Avg: 20	40

❖ Main con: Short jobs may wait a lot, aka "Convoy Effect"
Scheduling Policy: SJF
❖ Shortest Job First
❖ Ranking by Job Length; no preemption
Example: If P1, P2, P3 of lengths 10,40,10 units arrive closely in that order



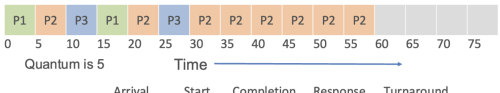
Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	10	0	10
P2	0	20	60	20	60
P3	0	10	20	10	20
		(FIFO Avg: 20 and 40)		Avg: 10	30

❖ Main cons: Job lengths might be unknown beforehand; possibly extensive completion times for long jobs (starvation)
Scheduling Policy: SCTF
❖ Shortest Completion Time First (aka SJF with preemption)
❖ Ranking by Job Length; preemption applied (arrival time indifferent)
Example: P1, P2, P3 of lengths 10, 40, 10 units arrive at different times



Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	10	10	20	0	10
P2	0	0	60	0	60
P3	25	25	35	0	10
		(SJF Avg: 10 and 30)		Avg: 0	26.7

❖ Main cons: Job lengths might be unknown beforehand; possibly extensive completion times for long jobs (starvation)
Concurrency
❖ Modern computers often have multiple processors and multiple cores per processor
❖ Concurrency: Multiple processors/cores run different/same set of instructions simultaneously on different/shared data
❖ New levels of shared caches are added
Scheduling Policy: Round Robin
❖ In Round Robin job lengths need not be known
❖ Fixed time *quantum* given to each job; cycle through jobs
Example: P1, P2, P3 of lengths 10,40,10 units arrive closely in that order



Process	Arrival t1	Start t2	Completion t3	Response t4 = t2-t1	Turnaround t5 = t3-t1
P1	0	0	20	0	20
P2	0	5	60	5	60
P3	0	10	30	10	30
		(SJF Avg: 10 & 30; SCTF Avg: 0 & 26.7)		Avg: 5	36.7

❖ RR is often very fair, but Avg Turnaround Time goes up!
Concurrency cont.
Multiprocessing: Different processes run on different cores (or entire CPUs) simultaneously
Thread: Generalization of OS's Process abstraction
❖ A program spawns many threads; each run parts of the program's computations simultaneously
❖ **Multithreading**: Same core used by many threads
❖ Issues in dealing with multithreaded programs that write shared data: Cache coherence, Locking; deadlocks, Complex scheduling
Scheduling for multiprocessing/multicore is more complex
Load Balancing: Ensuring that different cores/processors are kept roughly equally busy, i.e., reduce **idle times**
Multi-queue multiprocessor scheduling (MQMS) is common
❖ Each processor/core has its own job queue
❖ OS moves jobs across queues based on load
Most data-intensive computations in data science do not need concurrent writes on shared data!
❖ Concurrent low-level ops abstracted away by libraries/APIs
❖ **Partitioning / replication** of data simplifies concurrency

---

Lecture 6: Filesystem and Data File

File: A persistent sequence of bytes that stores a logically coherent digital object for an application
❖ **File Format**: An application-specific standard that dictates how to interpret and process a file's bytes
- Numerous file formats exist (e.g., TXT, DOC, GIF, MPEG); varying data models/types, domain-specific, etc.
❖ **Metadata**: Summary or organizing info about file content (aka *payload*) stored with file itself; format-dependent
Directory: A cataloging structure with a list of references to files and/or (recursively) other directories
❖ Typically treated as a *special kind of file.*
❖ Common terms: Subdirectory, Parent direc, Root directory.
Filesystem: The part of OS that helps programs create, manage, and delete files on disk (secondary storage)
❖ Roughly split into *logical level* and *physical level*
- Logical level exposes file and directory abstractions and offers System Call APIs for file handling
- Physical level works with disk firmware and moves bytes to/from disk to DRAM
❖ Dozens of filesystems exist, e.g., ext3, btrfs, NTFS, APFS
❖ Differ on how they layer file and directory abstractions as bytes, what metadata is stored, etc.; how data integrity/reliability is assured, support for editing/resizing, compression/encryption, etc.
❖ Some can work with (can be "**mounted**" by) multiple OSs.
Virtualization of File on Disk
OS abstracts a file on disk as a virtual object for processes
File Descriptor: An OS-assigned positive integer identifier /reference for a file's virtual object that a process can use
❖ 0/1/2 reserved for STDIN/STDOUT/STDERR
❖ **File Handle**: A programming language's abstraction on top of a file descriptor (fd)
❖ open(): Create a file; assign fd; optionally overwrite
❖ read(): Copy file's bytes on disk to in-mem. buffer; sized
❖ write(): Copy bytes from in-mem. buffer to file on disk
❖ fsync(): "Flush" (force write) "dirty" data to disk
❖ close(): Free up the fd and other OS state info on it
❖ lseek(): Position offset in file's fd (for random R/W later)
❖ Dozens more (rename, mkdir, chmod, etc.)
Files vs. Databases: Data Model
Database: An organized collection of interrelated data
❖ **Data Model**: An abstract model to define organization of data in a formal (mathematically precise) way
- For example: Relations, XML, Matrices, DataFrames
Every database is just an abstraction on top of data files!
❖ **Logical level**: Data model for higher-level reasoning
❖ **Physical level**: How bytes are layered on top of files
❖ All data systems (RDBMSs, Dask, Spark, TensorFlow, etc.) are application/platform software that use OS System Call API for handling data files
Structured Data: A form of data with regular substructure
❖ Typically serialized as restricted ASCII text file (TSV, CSV, etc.); Matrix/tensor as binary too; Can layer on Relations too!
❖ Can layer on Relations, Matrices, or DataFrames, or be treated as first-class data model
❖ Inherits flexibility in file formats (text, binary, etc.)
Relation Relational DB: Relations are serialized by most RDBMSs and Spark as binary file(s); often compressed
***Q:** What is the difference between Relation, Matrix, and DataFrame?*
❖ **Ordering**: Matrix and DataFrame have row/col numbers; Relation is orderless on both axes!
❖ **Schema Flexibility**: Matrix cells are numbers. Relation tuples conform to pre-defined schema. DataFrame has no pre-defined schema but all rows/cols can have names; col cells can be mixed types!
❖ **Transpose**: Supported by Matrix & DataFrame, not Relation
Semistructured Data: A form of data with less regular / more flexible substructure than structured data(Tree-Structured, Graph-Structured)
❖ Typically serialized as restricted ASCII text file (extensions XML, JSON, YML, etc.) for tree; Typically serialized with JSON or similar textual formats for graph
❖ Some data systems also offer binary file formats
❖ Can layer on Relations too
Data Files on Data "Lakes"
❖ **Data "Lake"**: Loose coupling of data file format for storage and data/query processing stack (vs RDBMS's tight coupling)
❖ JSON for raw data; Parquet processed is common
Pros and cons of Parquet v text-based files (CSV, JSON):
❖ **Less storage**: Parquet stores in compressed form; can be much smaller (even 10x); less I/O to read
❖ **Column pruning**: Enables app to read only columns needed to DRAM; even less I/O now!

❖ **Schema on file**: Rich metadata, stats inside format itself
❖ **Complex types**: Can store them in a column
❖ **Human-readability**: Cannot open with text apps directly
❖ **Mutability**:Parquet is immutable/read-only;no in-placeedits
❖ **Decompression/Deserialization overhead**: Depends on application tool; can go either way
❖ **Adoption in practice**: CSV/JSON support more pervasive but Parquet is catching up

Data as File: Other Common Formats

❖ **Machine Perception** data layer on tensors and/or time-series
❖ Myriad binary formats, typically with (lossy) compression, e.g., WAV for audio, MP4 for video, etc.
❖ **Text File** (aka plaintext): Human-readable ASCII characters
❖ **Docs/Multimodal File**: Myriad app-specific rich binary formats

Main Memory Management

Virtualization of DRAM with Pages

Page: An abstraction of fixed size chunks of memory/storage
❖ Makes it easier to virtualize and manage DRAM
❖ Usually OS configuration parameter; e.g., 4 KB – 16 KB
Page Frame: Virtual slot in DRAM to hold a page's content
OS **Memory Management** is a system with mechanisms to:
❖ Identify pages uniquely
❖ Read/write page from/to disk when requested by a process

Apportioning of DRAM: Elements

A process's **Address Space:**
❖ Slice of virtualize DRAM assigned to it alone!
❖ OS "translates" DRAM vs disk address

Page Replacement Policy:
❖ When DRAM fills up, which cached page to evict?
❖ Many policies in OS literature

Memory Leaks:
❖ Process forgot to "free" pages used a while ago
❖ Wastes DRAM and slows down system

Garbage Collection:
❖ Some PL implementations can auto-reclaim some wasted memory

Storing Data In Memory

Any data structure in memory is overlaid on pages; Process can ask OS for more memory in System Call API; If OS denies, process may crash
❖ **Apache Arrow**: One standard for columnar in-memory data layout; Compatible with Pandas, (Py)Spark, Parquet, etc.

Persistent Data Storage

❖ **Volatile Memory**: A data storage device that needs power/electricity to store bits; e.g., DRAM, CPU caches (SRAM)
❖ **Persistence**: Program state/data is available intact even after process finishes
❖ **Non-Volatile** or **Persistent** memory/storage: A data storage device that retains bits intact after power cycling
❖ E.g., all levels below DRAM in memory hierarchy
❖ "**Persistent Memory (PMEM)**": Marketing term for large DRAM that is backed up by battery power!
❖ **Non-Volatile RAM (NVRAM)**: Popular term for DRAM-like device that is genuinely non-volatile (no battery)

Disks: Aka secondary storage; likely holds the vast majority of the world's day-to-day business-critical data!
❖ Data storage/retrieval units: **disk blocks** or **pages**
❖ Unlike RAM, different disk pages have different retrieval times based on location:
❖ Need to *optimize* layout of data on disk pages
❖ Orders of magnitude performance gaps possible

Data Organization on Disk

Disk space is organized into files; Files are made up of disk pages aka blocks
Typical disk block/page size: 4KB or 8KB
❖ Basic unit of reads/writes for a disk
❖ OS/RAM page is not the same as disk page!
Typically, [OS/RAM page size] = [Disk page size] but not always; disk page can be a multiple, e.g., 1MB
❖ File data (de-)allocated in increments of disk pages

Magnetic Disk Quirks

❖ Key Principle: Sequential vs. Random Access Dichotomy
❖ Accessing disk pages in sequential order gives higher throughput
❖ Random reads/writes are OOM slower!
❖ Need to carefully lay out data pages on disk
❖ Abstracted away by data systems: Dask, Spark, RDBMSs

Flash SSD vs. Magnetic Hard Disks

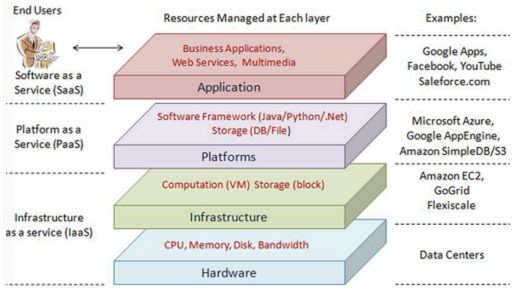
Roughly speaking: Flash combines the speed benefits of DRAM with persistence of disks
❖ Random reads/writes are not much worse
- Different locality of reference for data/file layout
- But still block-addressable like HDDs
❖ Data access latency: 100x faster!
❖ Data transfer throughput: Also 10-100x higher
❖ Parallel read/writes more feasible
❖ Cost per GB is 2-5x higher (a few years ago, range was 5-15x)
❖ Read-write impact asymmetry; much lower lifetimes

NVRAM vs. Magnetic Hard Disks

Roughly speaking: NVRAM is like a non-volatile form of DRAM, but with similar capacity as SSDs
❖ Random R/W with less to no SSD-style wear and tear
- Byte-addressability (not blocks like SSDs/HDDs)
- Spatial locality of reference like DRAM; radical change!
❖ Latency, throughput, parallelism, etc. similar to DRAM
❖ Alas, yet to see light of day in production settings
❖ Cost per GB: No one knows for sure yet!

Lecture 7: Cloud Computing

❖ Compute, storage, memory, networking, etc. are virtualized and exist on remote servers; rented by application users
❖ Main pros of cloud vs on-premise clusters:
- **Manageability**: Managing hardware is not user's problem
- **Pay-as-you-go**: Fine-grained pricing economics based on actual usage (granularity: seconds to years!)
- **Elasticity**: Can dynamically add or reduce capacity based on actual workload's demand
❖ Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), Software-as-a-Service (SaaS)



Evolution of Cloud Infrastructure

Data Center: Physical space from which a cloud is operated
❖ 3 generations of data centers/clouds:
❖ Cloud 1.0 (Past): Networked servers; user rents servers (time-sliced access) needed for data/software
❖ Cloud 2.0: "Virtualization" of networked servers; user rents amount of resource capacity; cloud provider has a lot more flexibility on provisioning (multi-tenancy, load balancing, more elasticity, etc.)
❖ Cloud 3.0: "Serverless" and disaggregated resources all connected to fast networks
❖ **Decoupling** of compute+memory from storage is common in cloud; Hybrids of shared-disk parallelism + shared-nothing parallelism; E.g, store datasets on S3 and read as needed to local EBS
❖ **Serverless** paradigm gaining traction for some applications, e.g., online ML prediction serving on websites
❖ User gives a program (function) to run and specifies CPU and DRAM needed; Cloud provider abstracts away all resource provisioning entirely; Higher resource efficiency; much cheaper, often by 10x vs Spot instances; Aka Function-as-a-Service (FaaS)
❖ Logical next step in serverless direction: full resource disaggregation! That is, compute, memory, storage, etc. are all network-attached and elastically added/removed

---

Lecture 8: Parallelism (Big data = volume, variety, velocity)

❖ Large-scale data is a game changer in data science:
❖ Enables **study of granular phenomena** in sciences, businesses, etc. not possible before
❖ Enables **new applications** and personalization/customization
❖ Enables more **complex ML prediction targets** and mitigates variance to offer **high accuracy**

Hardware has kept pace to power the above:
❖ Storage capacity has exploded (PB clusters)
❖ Compute capacity has grown (multi-core, GPUs, etc.)
❖ DRAM capacity has grown (10GBs to TBs)
❖ Cloud computing is "democratizing" access to hardware; SaaS
❖ Common in parallel data processing: "**threads**"
- Generalization of **process** abstraction of OS
❖ A program/process can spawn many threads
- Each runs its part of program's computations simultaneously
- All threads share address space (so, data too)
❖ In multi-core CPUs, a thread uses up 1 core
- "**Hyper-threading**": Virtualizes a core to run 2 threads!

Task Parallelism

❖ **Topological sort** of tasks in task graph for scheduling
❖ Notion of a "worker" can be at processor/core level, not just at node/server level
- Thread-level parallelism possible instead of process-level
- E.g., Dask: 4 worker nodes x 4 cores = 16 workers total
❖ Main pros of task parallelism:
- Simple to understand; easy to implement
- Independence of workers => low software complexity
❖ Main cons of task parallelism:
- Data replication across nodes; wastes memory/storage
- Idle times possible on workers
❖ **Degree of Parallelism**: The largest amount of concurrency possible in the task graph, i.e., how many tasks can be run simultaneously.
❖ **Speedup** = Completion time given only 1 worker / (divide) Completion time given n (>1) workers
❖ **Scaleup** : The ability of a system to retain the same performance ratio of tasks-per-resources when both the tasks and the resources increase at same rate.

Lecture 9&10:

❖ Modern machines often have multiple processors and multiple cores per processor; hierarchy of shared caches
❖ OS Scheduler now controls what cores/processors assigned to what processes/threads when tasks ready to run.
❖ **Single-Instruction Multiple-Data (SIMD)**: A fundamental form of parallel processing in which different chunks of data are processed by the "same" set of instructions shared by multiple processing units (PUs)
❖ **Single-Instruction Multiple Thread (SIMT)**: Generalizes notion of SIMD to different threads concurrently doing so
- Each thread may be assigned a core or a whole PU
❖ **Single-Program Multiple Data (SPMD)**: A higher level of abstraction generalizing SIMD operations or programs
- Under the hood, may use multiple processes or threads
- Each chunk of data processed by one core/PU
- Applicable to any CPU, not just vectorized PUs
- Most common form of parallel programming
❖ In data science computations, an often useful surrogate for completion time is the instruction throughput: FLOPs, i.e., number of floating point operations per second.
❖ Data processing programs may run GFLOPs (109 FLOPs).
❖ Nowadays, even home user hardware (like higher-end graphics cards, gaming machines) touches TFLOP performance.

Amdahl's Law: Formula to upper bound possible speedup
❖ A program has 2 parts: one that benefits from multi-core parallelism and one that does not
❖ Non-parallel part could be for control, memory stalls, etc

Moore's Law: The number of transistors in a dense integrated circuit doubles about every two years.

Dennard Scaling: As transistors get smaller, their power density stays constant, so that the power use stays in proportion with area.

Hardware Accelerators:

Graphics Processing Unit (GPU): Tailored for matrix/tensor ops
❖ Basic idea: use tons of ALUs; massive data parallelism (SIMD on steroids); now A100 offers ~20 TFLOPS for FP32!

Tensor Processing Unit (TPU): An "application-specific integrated circuit" (ASIC) created by Google in mid 2010s for the AlphaGo program (dedicated system for the Go board game). Used in even more specialized tensor ops in DL inference

Field-Programmable Gate Array (FPGA): Configurable for any class of programs; ~0.5-3 TFLOP/s
❖ Cheaper; new hardware-software integrated stacks for ML/DL

Basic Idea: Divide-and-conquer again. "Split" a data file (virtually or physically) and stage reads of its pages from disk to DRAM; vice versa for writes

4 key regimes of scalability / staging reads:
❖ **Single-node disk**: Paged access from file on local disk
❖ **Remote read**: Paged access from disk(s) over a network
❖ **Distributed memory:** Data fits on a cluster's total DRAM
❖ **Distributed disk**: Use entire memory hierarchy of cluster
❖ **Caching**: Retaining pages read from disk in DRAM
❖ **Eviction**: Removing a page frame's content in DRAM
❖ **Spilling**: Writing out pages from DRAM to disk
❖ If a page in DRAM is "**dirty**" (modified but not yet written back to disk; for example, if some bytes have been written to page), then eviction requires a spill; o/w, ignore that page
❖ The set of DRAM-resident pages typically changes over the lifetime of a process
❖ **Cache Replacement Policy**: The algorithm that chooses which page frame(s) to evict when a new page has to be cached but the OS cache in DRAM is full
❖ Typical frame ranking criteria:
➢ recency of use
➢ frequency of use
➢ number of processes reading it
❖ Typical optimization goal: Reduce total page I/O costs
❖ A few well-known policies:
❖ Least Recently Used (LRU): Evict page that was used longest ago
❖ Most Recently Used (MRU): (Opposite of LRU)

Scaling with Remote Reads

Basic Idea: Split data file (virtually or physically) and stage reads of its pages from disk to DRAM (vice versa for writes)
❖ Similar to scaling to local disk but not "local": Stage page reads from remote disk/disks over the network; More restrictive than scaling with local disk, since spilling is not possible or requires costly network I/Os; OK for a one-shot filescan access pattern
❖ Use DRAM to cache; dependency on replacement policies

❖ Can also use smaller local disk as cache

How many bits are needed at least
to distinguish 10000 unique data items?

Hints

We want the minimum number for bits x such that: 10000 < 2^x

2^{10} = 1024
2^{12} = 4096
2^{13} = 8192
2^{14} = 16384

From the above investigation we see that we need at least 14 bits.

Display 1000(base10) as a hexadecimal.

Finding the answer

We want to find the hexadecimal digits x, y, and z that represent 1000(base10) as xyz(base16).

These digits should satisfy
16^2 x+ 16^1 y + 16^0 z = 1000, or 256x + 16y + z = 1000.

The highest value of x so that 256x <= 1000 is 3: 256*3 = 768
This leaves a remainder of 1000-768 = 232 to be represented by 16y + z.

The highest value of y so that 16y <= 232 is 14: 16*14 = 224.
Remember that 14(base10) = E(base16)

This leaves a remainder 232-224 = 8 that is the value of z.

Therefore 1000(base10) = 3E8(base16) or 0x3E8.

Assume a custom float representation by using 6 bits

1	1	0	1	1	0
5	4	3	2	1	0

Bit index:

Decimal Interpretation: $(-1)^{sign} \times 2^{exponent-2} \times (1 + \sum_{i=1}^{3} b_{i-1}\, 2^{-i}) =$
$(-1)^{sign} \times 2^{exponent-2} \times (1 + b_0 2^{-1} + b_1 2^{-2} + b_2 2^{-3})$

where
sign: the first bit from the left (bit index 4)
exponent: the decimal value of the 2 bits in the next couple of positions (bit indices 4, 3).
Fraction component: Each one of the bits in the remaining index positions k, where k = 0, 1, 2.

The k-th bit is the value of the coefficient bk in the above equation.
What is the decimal representation of the custom 6-bit float sequence A=110110 ?
Let us call X the unknown decimal number. In the sequence A:
sign = 1
exponent = 10(base2) = 2(base10),
b_0 = 0, b_1 = 1, b_2 = 1. Therefore:

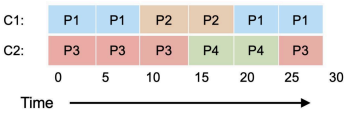
Then X = $-1^1 * 2^{2-2} * (1 + 0*2^{-1} + 1*2^{-2} + 1*2^{-3})$ =
= -1 * 1 * (1 + 0 + 0.25 + 0.125) =
= -1 * (1 + 0.375) = -1 * 1.375 = -1.375

	sign	exponent		fraction		
base-3 float representation :	x	x	x	x	x	x
Bit index :	5	4	3	2	1	0

Decimal Interpretation : $(-1)^{sign} \times 3^{exponent-4} \times (1 + \sum_{i=1}^{3} d_{3-i} 3^{-i})$

Let us call X the decimal number we seek.
For the base3 representation of the sequence 221022, the interpretation factors give:
- sign = 2(base3) = [2*3^0](base10) = 2(base10)
- exponent = 21(base3) = [2*3^1 + 1*3^0](base10) = [2*3 + 1](base10) = 7(base10)
- For the factor with the sum, we have the remaining 3 bits 022 with indices 2, 1, 0, so
  d_2 = 0(base3) = 0(base10)
  d_1 = 2(base3) = 2(base10)
  d_0 = 2(base3) = 2(base10)
  and the factor with the sum gives the decimal interpretation:
  1 + d_2*3^{-1} + d_1*3^{-2} + d_0*3^{-3} = 1 + 0*(1/3) + 2*(1/9) + 2*(1/27) = (27+6+2)/27 = 35/27

Eventually, we replace the above findings in the decimal representation expression:
X = (-1)^2 * 3$^{(7-4)}$ * (35/27) = 1 * 3^3 * (35/27) = 27 * (35/27) = 35



Consider the above Gantt Chart, same for all questions that follow. The chart shows the full concurrent schedule of
- 4 processes
- on a 2-core (cores C1 and C2) machine.

We are informed that:
- Processes P1 and P3 arrived at time t=0
- Process P2 arrived at time t=10
- Process P4 arrived at time t=15

1. What is the response time for P2?
   [Start] – [Arrival] = 10 – 10 = 0

2. What is the response time for P3?
   [Start] – [Arrival] = 0 – 0 = 0

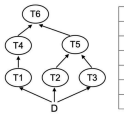
3. What is the average response time?
   Response time for both P1 and P4 is 0. Therefore, the average is 0.

4. What is the turnaround time for P1?
   [Completion] – [Arrival] = 30 – 0 = 30

5. What is the turnaround time for P4?
   [Completion] – [Arrival] = 25 – 15 = 10

6. Which scheduling policy is most likely to produce the present schedule?
   A. FIFO B. SJF C. SCTF D. Round Robin with a quantum of 5 E. None of the rest.

Notes:
A. Clearly, not a first-in first-out implementation.
B. SJF is *non-preemptive*; this means a scheduler **does not interrupt** a running job to start another (https://en.wikipedia.org/wiki/Shortest_job_next). Clearly, SJF not implemented.
C. SCTF is *preemptive*; this means a scheduler **will interrupt** a running job to let another one with faster completion time run first. In this example, P3 was interrupted to allow P4 to run. Yet when P4 arrived, P3 had half the length of P4 remaining to complete. Under SCTF, P3 should be prioritized and not interrupted, so not a SCTF implementation.
D. Clearly not a Round Robin with a quantum of 5.
E is the correct answer.



Task	Length
T1	10
T2	15
T3	10
T4	20
T5	10
T6	5

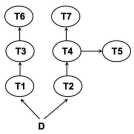
1. What is the largest degree of parallelism, if you were to execute this workload using task parallelism?
Answer: 3 (Because all tasks T1, T2, and T3 can run in parallel.)

2. Suppose you are given 3 identical worker nodes and use task parallelism. What is the lowest possible completion time of this workload?
Answer: 35 (We need to find the longest path from D to end, which is T1 →T4 →T6.)

3. Continuing with the above question's setup, what is the highest possible speedup against running on only one worker node?
Answer: 2x (Total time of all tasks on a single node is 70; thus, speedup is 70/35 = 2x.)

4. Continuing with the above question's setup, in a task-parallel schedule that offers the highest possible speedup, what is the highest possible idle time of one worker among the 3? Assume all workers on, from start to the end of the whole workload.
Answer: 25 (T1, T4, T6 on W1; T2 and T5 on W2; T3 on W3. Idle time is largest on W3: 35−10= 25.)

5. Now suppose you are given only 2 identical worker nodes for task parallelism. What is the lowest possible completion time of this workload now?
Answer: 40 (With only 2 workers, one best possible schedule places T3 after T2 and before T5 on W2, while W1 runs T1 and T4. Note that T6 cannot start until both T4 and T5 finish. So, new overall lowest completion time possible is 35 (T2, T3, T5 on W2) + 5 (for T6 on any of the workers).)



Task	Length
T1	15
T2	5
T3	10
T4	x
T5	10
T6	10
T7	10

Consider the above task graph and given task lengths (in time units). The value of x is a non-negative number that can vary as specified in each of the following questions.

1. What is the degree of parallelism of this workload for task-parallel execution?
Answer: 3 (The figure shows that tasks T5, T6, and T7 can be executed independently).

2. Assume x=20. Suppose the workload is executed in a task-parallel manner for the lowest possible completion time on 3 workers. All workers are on until the last task finishes. What is the total idle time (add across workers)?
Answer: If we have 3 workers: T1-T3-T6 and T2-T4-T7 can run in parallel on 2 workers, followed by T5 running on the 3rd worker. T5 needs to wait for 25 units to begin, and so do T6 and T7. T5, T6, and T7 finish at the same time. So, total idle time is the wait time for T1-T3 or T2-T4, which is 25.

3. Assume x=20. What is the lowest possible completion time of this workload with task-parallelism?
Answer: We need to find the longest single path for a worker. In the present scenario, it will take for all workers 35 units to complete their tasks.

4. Assume x=5. What is the value of the lowest possible completion time with task-parallelism when given only 2 workers?
Answer: If we only have 2 workers, then the path T1-T3-T6 takes 35 units for one worker; the path T2-T4-T7 (or T2-T4-T5) takes 20 units for the second worker. The second worker will finish earlier and can do the remaining task T5 (or T7) within 10 more units, thus totaling 30 units. As a result, the lowest possible completion given 2 workers will take 35 units, because this is the longest path needed to be followed (the one done by the first worker).

8. For each of the following answer True of False:

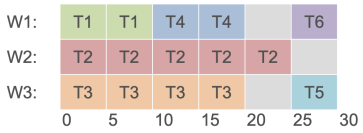
A. Databases are typically stored as files.	A. T
B. An OS typically has mechanisms to wrest control of hardware back from a user process.	B. T
C. More accurate ML models are always larger in the number of bytes of memory they need than less accurate ones.	C. F
D. SCTF is the fairest scheduling policy we discussed in class.	D. F
E. A filesystem is a specific format of serializing data files.	E. F
F. CPU caches are usually cheaper per MB than Flash SSD.	F. F
G. If DRAM is infinite Pareto frontiers are irrelevant in ML practice.	G. F
H. DataFrame and Relation are equivalent data models.	H. F

10. Suppose an SQL query takes 20min to run on a single worker node and *x* min when run on 5 worker nodes. What is the speedup for the given value of *x*? Is the speedup linear, sublinear, or superlinear?

A. x = 7min
B. x = 4min
C. x = 3min

A. Speedup = 20/7 = 2.86x; < 5x, sublinear
B. Speedup = 20/4 = 5x; = 5x, linear
C. Speedup = 20/3 = 6.67x; > 5x, superlinear

11. What is the speedup yielding by this task-parallel schedule on 3 workers against 1 worker?



3-worker time = 30 (from chart)
1-worker time =
10+25+20+10+5+5 = 75
Speedup = 75 / 30 = 2.5x

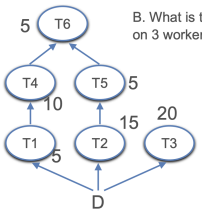
A 2 D 2.67
B 2.33 E 3
C 2.5

12. Consider the same Gantt Chart as in Ex. 11. Suppose you are given that T6 and T5 depend on T2. What will be the new speedup with 3 more workers for task-parallel execution? Explain your answer succinctly.

Speedup will still be only 2.5x.
T1, T2, and T3 can start in parallel; T4 can run after T1 on that same worker; but T5 and T6 can not start anywhere until T2 finishes. But once T2 finishes, T5 and T6 can run in parallel. So, Gantt chart even with 6 workers ends up looking the same/similar, with same completion time of 30.

13. Consider the following task graph with the task lengths shown. You are given 3 workers to execute this graph in a task-parallel manner like discussed in class.

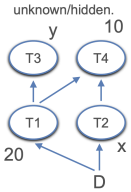
A. What is the lowest possible completion time of this workload?
25; it is the longest path = 15 + 5 + 5 = 25



B. What is the highest possible speedup of this workload on 3 workers vs its runtime on just 1 worker without idling?
2.4x; total 1-worker time = 5+15+20+10+5+5 = 60
Regardless of # workers, lowest possible compl. time is 25 as above.
So, highest possible speedup = 60/25 = 2.4x

C. What is the total idle time across all workers in a schedule that yields the highest speedup?
15; with 3 workers, we can hit above max speedup, with completion time of 25.
Total time consumed by 3 workers = 3 x 25 = 75
So, total idle time = 75 - 60 = 15

14. Consider the following task graph with some task lengths shown and some unknown/hidden.



A. What is the degree of parallelism in this task graph?
2; only 2 tasks can run concurrently based on given graph

B. You are given that the lowest possible completion time of this workload is 30. What are the maximum possible values of x and y?
y = 10, x = 20. Lowest possible compl. time is longest path, i.e., max (20+y, 20+10, x+10). Set it to 30. So, max value of y = ~~30-10 = 20~~ 30-20 = 10; likewise, max value of x = 30-10 = 20.

C.**(Advanced; Optional)** Instead of B, you are now given that y = 2x. What is the highest possible speedup for this workload with task-parallel execution?
1.5x; lowest possible comp. time now is max (20+2x, 20+10, x+10). To get highest possible speedup, you need its lowest possible value, which is just 30. That means 20+2x can be max 30, which means x can be max 5.
1-worker time = x+y+30 = 3*(x+10). So, max speedup is 3*(x+10)/30.
Setting x=5 then gives us 3*15/30 = 1.5x